

amuguna

개인 프로필 기반 공공 금융정보 매칭 서비스

제출 목적	공공데이터포털(data.go.kr) 오픈API 활용신청 및 운영계정 전환 신청 증빙
서비스명	amuguna (아무거나)
출품 대회	2026 금융 AI Challenge — 금융보안원 주최 / 금융위원회 후원 / 데이콘 운영
활용 유형	비영리 · 공모전 출품용 웹서비스 (개인/팀 개발)
기술 스택	TypeScript (Node.js 배치 + Next.js 웹) / PostgreSQL + pgvector
소스 저장소	github.com/ofseo03/amuguna
담당자 연락처	ofseo03@gmail.com
작성일	2026-08-11

본 문서는 신청 API를 실제로 어떻게 호출하고 처리하는지를 코드와 데이터 계약으로 제시하기 위해 작성되었습니다. 수집기 소스코드 전문과 API 응답 처리 구조, 호출량 산정 근거를 포함합니다.

1. 활용 목적

정부·지자체·공공기관이 제공하는 지원금·정책자금·금융상품 정보는 부처별 사이트에 흩어져 있고, 자격요건은 "만 19~34세, 중위소득 150% 이하, 무주택, 해당 시군 6개월 이상 거주" 같은 조합형입니다. 그 결과 받을 수 있는데 몰라서 신청하지 못하는 사각지대가 발생합니다.

본 서비스는 이용자가 입력한 **인적사항 5필드**(연령·성별·직업·거주지·소득분위)와 **"원하는 것" 한 줄**을 받아, 두 축의 교집합으로 정보를 좁힙니다.

축	질문	입력	판정 방식
자격	내가 대상인가?	인적사항 5필드	정형 조건 대조 (SQL)
의도	내가 찾는 것인가?	원하는 것 한 줄	의미 유사도 (벡터 검색)

자격만 맞추면 수백 건의 무관한 정보가 쏟아지고, 의도만 맞추면 대상이 아닌 것을 권하게 됩니다. 두 축이 서로 다른 것을 측정하기 때문에 **교집합이 곧 정보**가 됩니다.

공공데이터 활용의 핵심

신청 API는 이 서비스의 **유일한 정보 원천**입니다. 임의로 생성하거나 추정한 정보는 노출하지 않으며, 모든 결과 카드에 **원문 출처 URL과 수집 시각**을 함께 표시하여 이용자가 원본을 직접 확인할 수 있게 합니다.

2. 활용신청 대상 오픈API

공공데이터포털 인증키 1개로 아래 데이터셋을 활용합니다.

2.1 지원금·복지 (자격요건 원천)

데이터셋 ID	명칭	활용 내용
15113968	행정안전부_대한민국 공공서비스(혜택) 정보	중앙부처·지자체·공공기관·교육청 혜택 전체의 목록과 상세를 수집해 매칭 대상 모집단을 구성
15090532	한국사회보장정보원_중앙부처복지서비스	상세조회에 지원대상·선정기준·신청방법 텍스트에서 자격요건을 추출
15108347	한국사회보장정보원_지자체복지서비스	지자체 단위 복지서비스. 거주지 기준 필터링의 원천

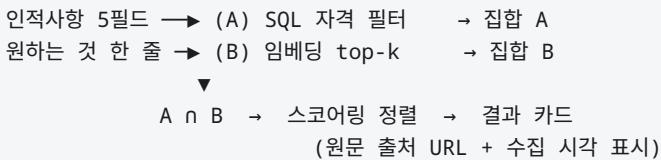
2.2 서민·정책금융 상품 (대회 주제 직결)

데이터셋 ID	명칭	활용 내용
15094787	금융위원회_서민금융상품기본정보	서민금융 대출·자산형성·사회적금융 상품의 지원대상·금리·한도·상환방식을 상품 카드로 제공
15106208	서민금융진흥원_대출상품한눈에 정보 서비스	자금용도·지원대상 기준 상품 매칭
15119396	한국주택금융공사_전세자금보증상품 추천서비스	전세보증금 관련 이용자에게 보증상품·지역별 최대임차보증금액 안내
15082039	한국주택금융공사_u-보증자리론대출정보	만기별 금리 정보
15074508	서민금융진흥원_서민대출상품 취급기관 정보	상품에서 실제 신청 창구로 연결
15037733	서민금융진흥원_서민금융통합지원센터 현황	디지털 취약계층 이용자에게 가까운 오프라인 창구를 함께 안내

2.3 확장

데이터셋 ID	명칭	활용 내용
15143273	한국고용정보원_온통청년_청년정책API	청년 대상 정책 보강
15125364	창업진흥원_K-Startup 조회서비스	창업·소상공인 지원사업 보강

15098547	한국부동산원_청약홈 분양정보 조회 서비스	주거 관련 공고 보강
15000115	법제처 국가법령정보 공유서비스	제도의 근거 법령 원문 링크



모든 조회는 사전 수집된 데이터베이스만 대상으로 합니다. 이용자 트래픽이 그대로 API 호출량으로 전가되지 않으므로 **포털 서비스에 부하를 주지 않으며**, 동시에 외부 장애로부터 서비스가 격리됩니다.

3.1 증분 수집 - 호출량 절감 설계

소스가 수정일 파라미터를 지원하면 전일 이후 변경분만 조회합니다. 지원하지 않으면 `content_hash` 비교로 변경분만 재처리합니다.

상황	판정	처리
external_id 없음	신규	원문 저장 → 파싱 → 임베딩 → 등록
있음 + 해시 동일	무변경	수집시각만 갱신. 추가 처리 없음
있음 + 해시 상이	수정	원문 이력 추가 → 재파싱 → 재임베딩 → 갱신
전량 대조에서 누락	종료	expired 처리 (행 삭제 없음)

content_hash 는 응답 전체가 아니라 **비교 대상 필드만**(제목·본문·기간·금액)으로 계산합니다. 조회수나 응답 생성 시각처럼 매 호출 바뀌는 휘발성 필드를 하시에 포함하면 전 건이 매일 "수정"으로 판정되어 증분 수집의 이점이 사라지기 때문입니다.

3.2 호출량 산정 (운영계정 전환 사유)

구분	산정
수집 주기	1일 1회 (심야 배치, GitHub Actions cron)
목록 조회	1회 100건 · 페이지네이션 — 소스당 일 5~50회
상세 조회	신규·수정 건에 한해 건당 1회
주 1회 전량 대조	ID 목록만 조회 (상세 호출 없음)
요청 간 간격	동일 도메인 1초 1요청, 동시 연결 1개
재시도	408/429/5xx에 한해 최대 2회, 지수 백오프 (250ms → 500ms)

중앙부처복지서비스(15090532) 운영계정 전환을 요청드리는 사유: 개발계정 한도 100회/일에서 상세조회를 건당 1회 사용하면 **하루 100건이 상한**이 되어, 초기 전량 적재에만 수십 일이 소요됩니다. 대회 심사 구간(2026-09-07 ~ 09-11)에 서비스가 정상 동작하려면 초기 적재가 그 전에 완료되어야 합니다.

4. 수집기 소스코드

공공데이터포털 API를 직접 호출하는 코드 전문입니다. 언어는 TypeScript(Node.js)입니다.

4.1 공통 수집기 — 호출·재시도·응답 검증

모든 수집기가 상속하는 기반 클래스입니다. 인증키 주입, 타임아웃, 재시도 정책, 응답 봉투(`resultCode`) 검증, 페이지네이션 종료 조건을 담당합니다. User-Agent에 서비스명과 연락처를 명시하여 포털 측에서 호출 주체를 식별할 수 있게 했습니다.

web/ingest/collectors/base.ts

268 lines · TypeScript

```
1  import { readFile } from "node:fs/promises";
2  import { resolve } from "node:path";
3
4  import { FIXTURES_DIR, settingsFromEnv, type Settings } from "../config";
5  import type { CollectedProgram } from "../models";
6
7  export const USER_AGENT =
8    "amuguna-ingest/0.1 (2026 금융 AI Challenge; contact: ofseo03@gmail.com)";
9
10 export class CollectorError extends Error {
11   constructor(message: string, options?: ErrorOptions) {
12     super(message, options);
13     this.name = "CollectorError";
14   }
15 }
16
17 export type FetchLike = (input: string | URL | Request, init?: RequestInit) => Promise<Response>;
18
19 export interface CollectorOptions {
20   settings?: Settings;
21   useFixtures?: boolean;
22   fetchImpl?: FetchLike;
23   pageSize?: number;
24   retries?: number;
25   timeoutMs?: number;
26 }
27
28 export interface FetchOptions {
29   since?: string | null;
30   maxPages?: number;
31 }
32
33 export type CollectorConstructor = new (options?: CollectorOptions) => Collector;
34
35 export abstract class Collector {
36   abstract readonly sourceKey: string;
37   abstract readonly endpoint: string;
38   abstract readonly idListEndpoint: string;
39   readonly defaultForm: string = "subsidy";
40
41   /**
42    * 인증키 발급처는 소스마다 다르다 (SPEC §3.5). 공공데이터포털 경우 소스는
43    * 계정당 키 1개를 공유하지만, 자체 창구에서 발급하는 소스는 자기 키를 쓴다.
44    * 기본값은 포털 키이며, 발급처가 다른 수집기가 재정의한다.
45    */
46   protected get apiKey(): string {
47     return this.settings.data_go_kr_api_key;
48   }
49
50   /** 미설정 오류 메시지에 쓸 환경변수 이름. `apiKey` 와 짝을 맞춰 재정의한다. */
```

```

51 protected readonly apiKeyEnvName: string = "DATA_GO_KR_API_KEY";
52
53 readonly settings: Settings;
54 readonly useFixtures: boolean;
55 readonly pageSize: number;
56 readonly retries: number;
57 readonly timeoutMs: number;
58 httpCalls = 0;
59
60 private readonly fetchImpl: FetchLike;
61
62 constructor(options: CollectorOptions = {}) {
63     this.settings = options.settings ?? settingsFromEnv();
64     this.useFixtures = options.useFixtures ?? false;
65     this.fetchImpl = options.fetchImpl ?? fetch;
66     this.pageSize = options.pageSize ?? 100;
67     this.retries = options.retries ?? 2;
68     this.timeoutMs = options.timeoutMs ?? 30_000;
69 }
70
71 protected abstract queryParams(options: {
72     since: string | null;
73     page: number;
74 }): Record<string, string | number>;
75
76 protected abstract items(payload: unknown): Record<string, unknown>[];
77
78 protected abstract mapItem(item: Record<string, unknown>): CollectedProgram | null;
79
80 get fixturePath(): string {
81     return resolve(FIXTURES_DIR, `${this.sourceKey}.json`);
82 }
83
84 private async loadJson(path: string): Promise<unknown> {
85     try {
86         return JSON.parse(await readFile(path, "utf8")) as unknown;
87     } catch (error) {
88         throw new CollectorError(`JSON 파일을 읽을 수 없음: ${path}`, { cause: error });
89     }
90 }
91
92 private async get(params: Record<string, string | number>): Promise<unknown> {
93     if (!this.apiKey) {
94         throw new CollectorError(`${this.sourceKey}: ${this.apiKeyEnvName} 미설정`);
95     }
96
97     const url = new URL(this.endpoint);
98     for (const [key, value] of Object.entries(params)) url.searchParams.set(key, String(value));
99
100     let lastError: unknown;
101     for (let attempt = 0; attempt <= this.retries; attempt += 1) {
102         this.httpCalls += 1;
103         try {
104             const response = await this.fetchImpl(url, {
105                 headers: { "User-Agent": USER_AGENT },
106                 redirect: "follow",
107                 signal: AbortSignal.timeout(this.timeoutMs),
108             });
109             if (!response.ok) {
110                 const retryable = response.status === 408 || response.status === 429 || response.status >= 500;
111                 if (!retryable || attempt === this.retries) {
112                     throw new CollectorError(`${this.sourceKey} 호출 실패: HTTP ${response.status}`);
113                 }
114                 lastError = new Error(`HTTP ${response.status}`);
115             } else {
116                 return (await response.json()) as unknown;
117             }

```

```

118     } catch (error) {
119         if (error instanceof CollectorError) throw error;
120         lastError = error;
121         if (attempt === this.retries) break;
122     }
123
124     await new Promise((done) => setTimeout(done, 250 * 2 ** attempt));
125 }
126
127 throw new CollectorError(`${this.sourceKey} 호출 실패`, { cause: lastError });
128 }
129
130 externalId(nativeId: string): string {
131     return `${this.sourceKey}:${nativeId}`;
132 }
133
134 private validateEmptyPage(payload: unknown): void {
135     if (!isRecord(payload)) {
136         throw new CollectorError(`${this.sourceKey}: 오류 또는 잘못된 응답 봉투`);
137     }
138
139     let valid = false;
140     let message = "";
141
142     if (this.sourceKey === "social_security") {
143         const response = isRecord(payload.response) ? payload.response : payload;
144         const header = recordOrUndefined(response.header);
145         const body = recordOrUndefined(response.body);
146         const pageItems = body?.items;
147         valid =
148             header !== undefined &&
149             String(header.resultCode) === "00" &&
150             (Array.isArray(pageItems) ||
151              (isRecord(pageItems) &&
152               (Object.keys(pageItems).length === 0 || Object.hasOwn(pageItems, "item"))));
153         message = header ? String(header.resultMsg || "") : "";
154     } else if (this.sourceKey === "bizinfo") {
155         const response = recordOrUndefined(payload.response) ?? {};
156         const header = recordOrUndefined(response.header) ?? {};
157         const body = recordOrUndefined(response.body) ?? {};
158         const pageItems = Object.hasOwn(payload, "jsonArray") ? payload.jsonArray : body.items;
159         const code = payload.resultCode ?? header.resultCode;
160         valid = String(code) === "0000" && (Array.isArray(pageItems) || isRecord(pageItems));
161         message = String(payload.resultMsg || header.resultMsg || "");
162     } else if (this.sourceKey === "finlife") {
163         const result = recordOrUndefined(payload.result);
164         valid = result !== undefined && String(result.err_cd) === "000" && Array.isArray(result.baseList);
165         message = result ? String(result.err_msg || "") : "";
166     } else {
167         return;
168     }
169
170     if (!valid) {
171         throw new CollectorError(
172             `${this.sourceKey}: 오류 또는 잘못된 응답 봉투${message ? `: ${message}` : ""}`,
173         );
174     }
175 }
176
177 async fetch({ since = null, maxPages = 5 }: FetchOptions = {}): Promise<CollectedProgram[]> {
178     const payloads: unknown[] = [];
179     if (this.useFixtures) {
180         payloads.push(await this.loadJson(this.fixturePath));
181     } else {
182         for (let page = 1; page <= maxPages; page += 1) {
183             const payload = await this.get(this.queryParams({ since, page }));
184             const pageItems = this.items(payload);

```

```

185         if (pageItems.length === 0) this.validateEmptyPage(payload);
186         payloads.push(payload);
187         if (pageItems.length < this.pageSize) break;
188     }
189 }
190
191 const collected: CollectedProgram[] = [];
192 const seen = new Set<string>();
193 for (const payload of payloads) {
194     for (const item of this.items(payload)) {
195         try {
196             const program = this.mapItem(item);
197             if (!program || seen.has(program.external_id)) continue;
198             seen.add(program.external_id);
199             collected.push(program);
200         } catch (error) {
201             console.warn(`${this.sourceKey}: 항목 매핑 실패`, error);
202         }
203     }
204 }
205 return collected;
206 }
207
208 async listExternalIds(): Promise<Set<string>> {
209     if (this.useFixtures) {
210         const idsPath = resolve(FIXTURES_DIR, `${this.sourceKey}.ids.json`);
211         try {
212             const payload = await this.loadJson(idsPath);
213             if (!isRecord(payload) || !Array.isArray(payload.ids)) {
214                 throw new CollectorError(`${this.sourceKey}: 잘못된 ID 픽스처`);
215             }
216             return new Set(payload.ids.map((id) => this.externalId(String(id))));
217         } catch (error) {
218             if (!(error instanceof CollectorError) || !error.cause || !isMissingFile(error.cause)) throw
error;
219         }
220     }
221     return new Set((await this.fetch({ maxPages: this.useFixtures ? 5 : 50 })).map((p) => p.external_id));
222 }
223 }
224
225 export function isRecord(value: unknown): value is Record<string, unknown> {
226     return typeof value === "object" && value !== null && Array.isArray(value);
227 }
228
229 export function recordOrUndefined(value: unknown): Record<string, unknown> | undefined {
230     return isRecord(value) ? value : undefined;
231 }
232
233 function isMissingFile(error: unknown): boolean {
234     return isRecord(error) && error.code === "ENOENT";
235 }
236
237 export function firstOf(
238     item: Record<string, unknown>,
239     keys: readonly string[],
240     fallback = "",
241 ): string {
242     for (const key of keys) {
243         const value = item[key];
244         if (value !== null && value !== undefined && value !== "") return String(value).trim();
245     }
246     return fallback;
247 }
248
249 export function parseAmount(text: string): [number | null, number | null] {
250     if (!text) return [null, null];

```

```

251 const units: Record<string, number> = {
252     억: 100_000_000,
253     천만: 10_000_000,
254     백만: 1_000_000,
255     만: 10_000,
256 };
257 const values = [...text.matchAll(/(\d[\d,.]*)\s*(억|천만|백만|만)?\s*원/g)].map(
258     (match) => Number.parseFloat(match[1].replaceAll(",", "")) * (units[match[2] ?? ""] ?? 1),
259 );
260 if (values.length === 0) return [null, null];
261 if (values.length === 1) {
262     const value = Math.trunc(values[0]);
263     if (/최대|이내|한도|까지/.test(text)) return [null, value];
264     if (/최소|이상|부터/.test(text)) return [value, null];
265     return [value, value];
266 }
267 return [Math.trunc(Math.min(...values)), Math.trunc(Math.max(...values))];
268 }

```

4.2 한국사회보장정보원 복지서비스 수집기

목록조회(`NationalWelfarelistV001`)를 호출하고, 응답 항목을 서비스 내부 모델로 변환합니다. `srchModDt` (수정일) 파라미터로 증분 수집하며, 응답 필드명이 케이스별로 다를 수 있어 후보 키를 순서대로 시도합니다(`firstOf`).

`web/ingest/collectors/social-security.ts`

97 lines · TypeScript

```

1  import { CollectedProgram } from "../models";
2  import { Collector, firstOf, isRecord, parseAmount, recordOrUndefined } from "../base";
3
4  function ymd(value: string): string | null {
5      const digits = [...value].filter((character) => /\d/.test(character)).join("");
6      return digits.length === 8
7          ? `${digits.slice(0, 4)}-${digits.slice(4, 6)}-${digits.slice(6, 8)}`
8          : null;
9  }
10
11 export class SocialSecurityCollector extends Collector {
12     readonly sourceKey = "social_security";
13     readonly endpoint =
14         "https://apis.data.go.kr/B554287/NationalWelfareInformationsV001/NationalWelfarelistV001";
15     readonly idListEndpoint = this.endpoint;
16     override readonly defaultForm = "subsidy";
17
18     protected queryParams({ since, page }: { since: string | null; page: number }) {
19         const params: Record<string, string | number> = {
20             serviceKey: this.apiKey,
21             callTp: "L",
22             pageNo: page,
23             numOfRows: this.pageSize,
24             srchKeyCode: "001",
25             _type: "json",
26         };
27         if (since) params.srchModDt = since.replaceAll("-", "");
28         return params;
29     }
30
31     protected items(payload: unknown): Record<string, unknown>[] {
32         if (!isRecord(payload)) return [];
33         const response = recordOrUndefined(payload.response);
34         const body = recordOrUndefined(response?.body) ?? recordOrUndefined(payload.body);
35         let items: unknown = body?.items ?? {};
36         if (isRecord(items)) items = items.item ?? [];
37         if (isRecord(items)) items = [items];

```

```

38     return Array.isArray(items) ? items.filter(isRecord) : [];
39 }
40
41 protected mapItem(item: Record<string, unknown>): CollectedProgram | null {
42     const nativeId = firstOf(item, ["servId", "servid", "SERV_ID"]);
43     const title = firstOf(item, ["servNm", "servnm", "SERV_NM"]);
44     if (!nativeId || !title) return null;
45
46     const outline = firstOf(item, ["wlfareInfoOutlCn", "servDgst", "servDtlLink"]);
47     const target = firstOf(item, ["sprtTrgtCn", "trgterIndvdlArray"]);
48     const criteria = firstOf(item, ["slctCritCn"]);
49     const benefit = firstOf(item, ["alwServCn"]);
50     const eligibility = [target, criteria].filter(Boolean).join("\n");
51     const bodyText = [
52         outline,
53         benefit && `[자원내용]\n${benefit}`,
54         eligibility && `[자원대상]\n${eligibility}`,
55     ]
56         .filter(Boolean)
57         .join("\n\n");
58
59     const issuer = firstOf(item, ["jurOrgNm", "jurMnofNm"], "정부");
60     const issuerLevel = ["부", "청", "처", "위원회", "공단"].some((key) => issuer.includes(key))
61         ? "central"
62         : ["특별시", "광역시", "도", "특별자치시", "특별자치도"].some((key) =>
63             issuer.endsWith(key),
64         )
65         ? "metro"
66         : "local";
67     const amountText = firstOf(item, ["sprtAmtCn", "alwServCn"]);
68     const [amountMin, amountMax] = parseAmount(amountText);
69     const startsAt = ymd(firstOf(item, ["enfcBgngYmd"]));
70     const endsAt = ymd(firstOf(item, ["enfcEndYmd"]));
71
72     return new CollectedProgram({
73         external_id: this.externalId(nativeId),
74         source_key: this.sourceKey,
75         source_url: firstOf(
76             item,
77             ["servDtlLink", "servUrl"],
78             `https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=${nativeId}`,
79         ),
80         title,
81         body_text: bodyText || title,
82         eligibility_text: eligibility,
83         form: this.defaultForm,
84         issuer,
85         issuer_level: issuerLevel,
86         benefit_amount_text: amountText,
87         benefit_amount_min: amountMin,
88         benefit_amount_max: amountMax,
89         apply_url: firstOf(item, ["applmetUrl", "onapPsbltYn"]),
90         apply_method: firstOf(item, ["applmetCn", "srvPvsnmNm"]),
91         starts_at: startsAt,
92         ends_at: endsAt,
93         is_always_open: endsAt === null,
94         raw_body: item,
95     });
96 }
97 }

```

4.3 지원사업 공고 수집기

```

1 import { CollectedProgram } from "../models";
2 import { Collector, firstOf, isRecord, parseAmount, recordOrUndefined } from "./base";
3
4 function splitPeriod(text: string): [string | null, string | null, boolean] {
5     if (!text) return [null, null, false];
6     const dates = [...text.matchAll(/(\d{4})[./-]?(\d{2})[./-]?(\d{2})/g)].map(
7         (match) => `${match[1]}-${match[2]}-${match[3]}`,
8     );
9     const always = /예산\s*소진|상시|연중/.test(text);
10    if (dates.length >= 2) return [dates[0], dates[1], false];
11    if (dates.length === 1) return [dates[0], null, true];
12    return [null, null, always];
13 }
14
15 export class BizinfoCollector extends Collector {
16     readonly sourceKey = "bizinfo";
17     readonly endpoint = "https://www.bizinfo.go.kr/uss/rss/bizinfoApi.do";
18     readonly idListEndpoint = this.endpoint;
19     override readonly defaultForm = "loan";
20
21     // 기업마당은 공공데이터포털 경유가 아니라 bizinfo.go.kr 자체 발급 키를 쓴다 (SPEC §3.5).
22     protected override readonly apiKeyEnvName = "BIZINFO_KEY";
23
24     protected override get apiKey(): string {
25         return this.settings.bizinfo_key;
26     }
27
28     protected queryParams({ since, page }: { since: string | null; page: number }): {
29         const params: Record<string, string | number> = {
30             crtfcKey: this.apiKey,
31             dataType: "json",
32             pageUnit: this.pageSize,
33             pageIndex: page,
34         };
35         if (since) params.searchBgnDe = since.replaceAll("-", "");
36         return params;
37     }
38
39     protected items(payload: unknown): Record<string, unknown>[] {
40         if (!isRecord(payload)) return [];
41         const response = recordOrUndefined(payload.response);
42         const body = recordOrUndefined(response?.body);
43         let items: unknown = payload.jsonArray ?? body?.items;
44         if (isRecord(items)) items = [items];
45         return Array.isArray(items) ? items.filter(isRecord) : [];
46     }
47
48     protected mapItem(item: Record<string, unknown>): CollectedProgram | null {
49         const nativeId = firstOf(item, ["pblancId", "pblance_id"]);
50         const title = firstOf(item, ["pblancNm", "pblanc_nm"]);
51         if (!nativeId || !title) return null;
52
53         const summary = firstOf(item, ["bsnsSumryCn", "bsns_sumry_cn"]);
54         const target = firstOf(item, ["trgetNm", "trget_nm"]);
55         const realm = firstOf(item, ["pldirSportRealmLclasCodeNm"]);
56         const bodyText = [
57             summary,
58             realm && `[지원분야] ${realm}`,
59             target && `[지원대상]\n${target}`,
60         ]
61             .filter(Boolean)
62             .join("\n\n");
63         const issuer = firstOf(item, ["jrsdInsttNm", "excInsttNm"], "중소벤처기업부");
64         const issuerLevel = [
65             "서울",
66             "부산",

```

```

67     "대구",
68     "인천",
69     "광주",
70     "대전",
71     "울산",
72     "세종",
73     "경기",
74     "강원",
75     "충청",
76     "전라",
77     "경상",
78     "제주",
79     "전북",
80     "전남",
81     "경북",
82     "경남",
83     "충북",
84     "충남",
85     ].some((prefix) => issuer.startsWith(prefix))
86     ? "metro"
87     : "central";
88     const benefitText = firstOf(item, ["sportScvlNm", "sportAmtCn"]);
89     const [amountMin, amountMax] = parseAmount(benefitText || summary);
90     const [startsAt, endsAt, always] = splitPeriod(firstOf(item, ["reqstBeginEndDe"]));
91
92     return new CollectedProgram({
93         external_id: this.externalId(nativeId),
94         source_key: this.sourceKey,
95         source_url: firstOf(
96             item,
97             ["pblancUrl", "rceptEngnHmpgUrl"],
98             `https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?pblancId=${nativeId}`,
99         ),
100         title,
101         body_text: bodyText || title,
102         eligibility_text: target,
103         form: this.defaultForm,
104         issuer,
105         issuer_level: issuerLevel,
106         benefit_amount_text: benefitText,
107         benefit_amount_min: amountMin,
108         benefit_amount_max: amountMax,
109         apply_url: firstOf(item, ["rceptEngnHmpgUrl", "pblancUrl"]),
110         apply_method: firstOf(item, ["reqstMthPapersCn", "rceptMthNm"], "온라인 신청"),
111         starts_at: startsAt,
112         ends_at: endsAt,
113         is_always_open: always && endsAt === null,
114         raw_body: item,
115     });
116 }
117 }

```

4.4 금융상품 공시 수집기

기본정보(`baseList`)와 옵션정보(`optionList`)를 상품코드로 결합하는 구조를 처리합니다.

`web/ingest/collectors/finlife.ts`

103 lines · TypeScript

```

1  import { CollectedProgram } from "../models";
2  import { Collector, firstOf, isRecord, parseAmount, recordOrUndefined } from "../base";
3
4  export const TOP_FIN_GRP_NO = "020000";
5
6  function formatRate(value: unknown): string {

```

```

7     return typeof value === "number" && Number.isInteger(value) ? value.toFixed(1) : String(value ?? "");
8 }
9
10 export class FinlifeCollector extends Collector {
11     readonly sourceKey = "finlife";
12     readonly endpoint = "https://finlife.fss.or.kr/finlifeapi/depositProductsSearch.json";
13     readonly idListEndpoint = this.endpoint;
14     override readonly defaultForm = "product";
15
16     // 금감원 금융상품통합비교공시는 finlife.fss.or.kr 자체 발급 키를 쓴다 (SPEC §3.5).
17     protected override readonly apiKeyEnvName = "FINLIFE_KEY";
18
19     protected override get apiKey(): string {
20         return this.settings.finlife_key;
21     }
22
23     protected queryParams({ page }: { since: string | null; page: number }) {
24         return {
25             auth: this.apiKey,
26             topFinGrpNo: TOP_FIN_GRP_NO,
27             pageNo: page,
28         };
29     }
30
31     protected items(payload: unknown): Record<string, unknown>[] {
32         if (!isRecord(payload)) return [];
33         const result = recordOrUndefined(payload.result) ?? {};
34         const base = Array.isArray(result.baseList) ? result.baseList : [];
35         const options = Array.isArray(result.optionList) ? result.optionList : [];
36         const byProduct = new Map<string, Record<string, unknown>[]>();
37         for (const option of options) {
38             if (!isRecord(option)) continue;
39             const code = String(option.fin_prdt_cd);
40             byProduct.set(code, [...(byProduct.get(code) ?? []), option]);
41         }
42         return base.filter(isRecord).map((row) => ({
43             ...row,
44             _options: byProduct.get(String(row.fin_prdt_cd)) ?? [],
45         }));
46     }
47
48     protected mapItem(item: Record<string, unknown>): CollectedProgram | null {
49         const productCode = firstOf(item, ["fin_prdt_cd"]);
50         const companyCode = firstOf(item, ["fin_co_no"]);
51         const productName = firstOf(item, ["fin_prdt_nm"]);
52         if (!productCode || !productName) return null;
53
54         const company = firstOf(item, ["kor_co_nm"], "금융회사");
55         const joinWay = firstOf(item, ["join_way"]);
56         const joinMember = firstOf(item, ["join_member"]);
57         const special = firstOf(item, ["spcl_cnd"]);
58         const note = firstOf(item, ["etc_note"]);
59         const deny = firstOf(item, ["join_deny"]);
60         const options = Array.isArray(item._options) ? item._options.filter(isRecord) : [];
61         const ratelines = options.map(
62             (option) =>
63             `- ${String(option.save_trm)}개월 ${String(option.intr_rate_type_nm ?? "")} 기본
64             ${formatRate(option.intr_rate)}% / 최고 ${formatRate(option.intr_rate2)}%`,
65             );
66         const bodyText = [
67             `${company}의 ${productName} 상품입니다.`,
68             joinWay && `[가입방법] ${joinWay}`,
69             ratelines.length && `[금리]\n${ratelines.join("\n")}`,
70             special && `[우대조건]\n${special}`,
71             note && `[유의사항]\n${note}`,
72             joinMember && `[가입대상]\n${joinMember}`,
73         ]

```

```

73     .filter(Boolean)
74     .join("\n\n");
75     const limitText = firstOf(item, ["max_limit"]);
76     let [amountMin, amountMax] = parseAmount(/^\d+$/ .test(limitText) ? `${limitText}원` : limitText);
77     if (/^\d+$/ .test(limitText)) {
78         amountMin = null;
79         amountMax = Number.parseInt(limitText, 10);
80     }
81
82     return new CollectedProgram({
83         external_id: this.externalId(`${companyCode}-${productCode}`),
84         source_key: this.sourceKey,
85         source_url: "https://finlife.fss.or.kr/finlife/svings/fdrmDpst/list.do?menuId=2000100",
86         title: `[${company}] ${productName}`,
87         body_text: bodyText,
88         eligibility_text: joinMember || deny,
89         form: this.defaultForm,
90         issuer: company,
91         issuer_level: "central",
92         benefit_amount_text: limitText ? `최고한도 ${limitText}원` : "",
93         benefit_amount_min: amountMin,
94         benefit_amount_max: amountMax,
95         apply_url: "https://finlife.fss.or.kr/",
96         apply_method: joinWay || "영업점·인터넷뱅킹",
97         starts_at: null,
98         ends_at: null,
99         is_always_open: true,
100        raw_body: item,
101    });
102 }
103 }

```

4.5 수집기 등록

web/ingest/collectors/index.ts

16 lines · TypeScript

```

1  import { BizinfoCollector } from "../bizinfo";
2  import type { CollectorConstructor } from "../base";
3  import { FinlifeCollector } from "../finlife";
4  import { SocialSecurityCollector } from "../social-security";
5
6  export { BizinfoCollector } from "../bizinfo";
7  export { Collector, CollectorError, firstOf, parseAmount } from "../base";
8  export type { CollectorConstructor, CollectorOptions, FetchLike, FetchOptions } from "../base";
9  export { FinlifeCollector, TOP_FIN_GRP_NO } from "../finlife";
10 export { SocialSecurityCollector } from "../social-security";
11
12 export const COLLECTORS: Record<string, CollectorConstructor> = {
13     social_security: SocialSecurityCollector,
14     bizinfo: BizinfoCollector,
15     finlife: FinlifeCollector,
16 };

```

4.6 설정 — 인증키 주입

인증키는 **소스코드에 포함하지 않고** 환경변수로만 주입합니다. 저장소에는 키가 커밋되지 않으며, 실행 환경(GitHub Actions Secrets)에서 공급합니다. 공공데이터포털 경우 소스는 `DATA_GO_KR_API_KEY` 하나를 공유하고, 자체 창구에서 키를 발급하는 소스는 각자의 환경변수(`BIZINFO_KEY`, `FINLIFE_KEY`)를 사용하여 **포털 인증키가 포털 외부로 전송되지 않도록** 분리했습니다.

web/ingest/config.ts

45 lines · TypeScript

```

1  import { dirname, resolve } from "node:path";
2  import { fileURLToPath } from "node:url";
3
4  export const PACKAGE_DIR = dirname(fileURLToPath(import.meta.url));
5  export const REPO_ROOT = resolve(PACKAGE_DIR, "../..");
6  export const SHARED_DIR = resolve(REPO_ROOT, "shared");
7  export const FIXTURES_DIR = resolve(REPO_ROOT, "ingest/fixtures");
8
9  export const EMBEDDING_DIM = 1024;
10 export const LLM_MODEL = "claude-sonnet-5";
11
12 export interface Settings {
13     database_url: string;
14     // 인증키는 발급처별로 다르다 (SPEC §3.5). 포털 키 하나로 세 소스를 부를 수 없다.
15     data_go_kr_api_key: string;
16     bizinfo_key: string;
17     finlife_key: string;
18     anthropic_api_key: string;
19     embedding_provider: string;
20     embedding_api_key: string;
21     mock_embeddings: boolean;
22 }
23
24 function envValue(env: NodeJS.ProcessEnv, name: string, fallback = ""): string {
25     return (env[name] || fallback).trim();
26 }
27
28 export function settingsFromEnv(env: NodeJS.ProcessEnv = process.env): Settings {
29     const provider = envValue(env, "EMBEDDING_PROVIDER", "mock").toLowerCase() || "mock";
30     const forcedMock = envValue(env, "MOCK_EMBEDDINGS") === "1";
31     const embeddingKey = envValue(env, "EMBEDDING_API_KEY");
32
33     return {
34         database_url: envValue(env, "DATABASE_URL"),
35         data_go_kr_api_key: envValue(env, "DATA_GO_KR_API_KEY"),
36         bizinfo_key: envValue(env, "BIZINFO_KEY"),
37         finlife_key: envValue(env, "FINLIFE_KEY"),
38         anthropic_api_key: envValue(env, "ANTHROPIC_API_KEY"),
39         embedding_provider: forcedMock ? "mock" : provider,
40         embedding_api_key: embeddingKey,
41         mock_embeddings: forcedMock || provider === "mock" || !embeddingKey,
42     };
43 }
44
45 export const loadSettings = settingsFromEnv;

```

5. API 응답 처리 계약 (JSON)

키 발급 전 개발·테스트를 위해 **API 응답 봉투 구조를 그대로 모사한 픽스처**를 사용했습니다. 실제 API를 호출하지 않고도 파싱·저장 로직을 검증하기 위한 것으로, 키 발급 후에는 실제 응답으로 필드 명세를 확정합니다.

5.1 복지서비스 응답 구조

ingest/fixtures/social_security.json

232 lines · JSON

```
1  {
2    "_fixture_note": "한국사회보장정보원 사회보장급여 목록 API 응답 '봉투 그대로' 흉내낸 픽스처. 필드명·구조는 W1 실물 검증 대상 (SPEC §3.3). inqNum(조회수)과 body.resultTime 은 매 호출 바뀌는 휘발성 필드로, content_hash 계산에서 제외되는지 확인하는 용도로 일부러 넣었다 (SPEC §3.2).",
3    "response": {
4      "header": { "resultCode": "00", "resultMsg": "NORMAL SERVICE." },
5      "body": {
6        "numOfRows": 12,
7        "pageNo": 1,
8        "totalCount": 12,
9        "resultTime": "2026-08-08T18:12:03+09:00",
10       "items": {
11         "item": [
12           {
13             "servId": "WII0000345",
14             "servNm": "청년월세 한시 특별지원",
15             "jurMnofNm": "국토교통부",
16             "jurOrgNm": "국토교통부",
17             "wlfareInfoOutlCn": "부모와 따로 거주하는 저소득 청년의 주거비 부담을 덜기 위해 월세를 한시적으로 지원하는 사업입니다.",
18             "sprtTrgtCn": "만 19세 이상 34세 이하 무주택 청년으로 부모와 별도 거주하는 자",
19             "slctCritCn": "청년 본인 가구 기준 중위소득 60% 이하이며, 원가구 기준 중위소득 100% 이하인 경우 선정합니다. 임차보증금 5천만원 이하 및 월세 60만원 이하 주택 거주자에 한합니다.",
20             "alwServCn": "월 최대 20만원을 12개월간 지원",
21             "sprtAmtCn": "월 최대 20만원",
22             "applmetCn": "복지로 온라인 신청 또는 주소지 행정복지센터 방문 신청",
23             "applmetUrl": "https://www.bokjiro.go.kr/",
24             "servDtllLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII0000345",
25             "enfcBgngYmd": "20260301",
26             "enfcEndYmd": "20261231",
27             "srvPvsnNm": "현금",
28             "inqNum": "184203"
29           },
30           {
31             "servId": "WII0000112",
32             "servNm": "주거급여(임차급여)",
33             "jurMnofNm": "국토교통부",
34             "jurOrgNm": "국토교통부",
35             "wlfareInfoOutlCn": "소득과 주거 형태를 고려해 임차료를 지원하여 주거 안정을 돕는 제도입니다.",
36             "sprtTrgtCn": "임차료를 부담하는 임차가구",
37             "slctCritCn": "기준 중위소득 48% 이하 가구를 대상으로 하며, 부양의무자 기준은 적용하지 않습니다. 실제 임차료를 지불하고 있어야 합니다.",
38             "alwServCn": "지역·가구원수별 기준임대로 범위에서 실제 임차료 지급",
39             "sprtAmtCn": "월 최대 63만원",
40             "applmetCn": "주민등록 주소지 행정복지센터 방문 신청",
41             "applmetUrl": "https://www.bokjiro.go.kr/",
42             "servDtllLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII0000112",
43             "enfcBgngYmd": "20260101",
44             "enfcEndYmd": "",
45             "srvPvsnNm": "현금",
46             "inqNum": "902114"
```

```

    },
    {
        "servId": "WII0000001",
        "servNm": "기초연금",
        "jurMnofNm": "보건복지부",
        "jurOrgNm": "보건복지부",
        "wlfareInfoOutlCn": "어르신 안정된 노후생활을 돕기 위해 매월 연금을 지급하는 제도입니다.",
        "sprtTrgtCn": "만 65세 이상 어르신 중 소득인정액이 선정기준액 이하인 분",
        "slctCritCn": "소득 하위 70%에 해당하는 어르신을 선정합니다. 공무원연금 등 직역연금 수급자는 원칙적으로 제외됩니다.",
        "alwServCn": "월 최대 34만원 지급",
        "sprtAmtCn": "월 최대 34만원",
        "applmetCn": "국민연금공단 지사 또는 행정복지센터 방문 신청, 복지로 온라인 신청",
        "applmetUrl": "https://www.bokjiro.go.kr/",
        "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII0000001",
        "enfcBgngYmd": "20260101",
        "enfcEndYmd": "",
        "srvPvsnNm": "현금",
        "inqNum": "1520932"
    },
    {
        "servId": "WII00000078",
        "servNm": "긴급복지지원 (생계지원)",
        "jurMnofNm": "보건복지부",
        "jurOrgNm": "보건복지부",
        "wlfareInfoOutlCn": "갑작스러운 위기상황으로 생계유지가 곤란한 가구에 생계비를 신속하게 지원합니다.",
        "sprtTrgtCn": "주소득자의 사망·질병·실직 등으로 소득을 상실한 위기가구",
        "slctCritCn": "기준 중위소득 75% 이하이면서 재산 및 금융재산 기준을 충족해야 합니다. 동일 사유로 중복 지원 불가합니다.",
        "alwServCn": "4인 가구 기준 월 187만원 이내 생계비 지급",
        "sprtAmtCn": "월 최대 187만원",
        "applmetCn": "시군구청 또는 보건복지상담센터(129) 신청",
        "applmetUrl": "https://www.bokjiro.go.kr/",
        "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII00000078",
        "enfcBgngYmd": "20260101",
        "enfcEndYmd": "",
        "srvPvsnNm": "현금",
        "inqNum": "331002"
    },
    {
        "servId": "WII00000254",
        "servNm": "노인맞춤돌봄서비스",
        "jurMnofNm": "보건복지부",
        "jurOrgNm": "보건복지부",
        "wlfareInfoOutlCn": "일상생활 영위가 어려운 취약 어르신에게 안전지원, 사회참여, 생활교육 등 맞춤형 돌봄을 제공합니다.",
        "sprtTrgtCn": "만 65세 이상 국민기초생활수급자, 차상위계층 또는 기초연금 수급자",
        "slctCritCn": "독거·조손 등 돌봄이 필요한 어르신을 우선 선정합니다. 장기요양 등급판정자는 제외합니다.",
        "alwServCn": "안전지원·사회참여·생활교육 등 서비스 제공",
        "sprtAmtCn": "",
        "applmetCn": "주소지 행정복지센터 방문 신청",
        "applmetUrl": "https://www.bokjiro.go.kr/",
        "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII00000254",
        "enfcBgngYmd": "20260101",
        "enfcEndYmd": "",
        "srvPvsnNm": "서비스",
        "inqNum": "88231"
    },
    {
        "servId": "WII00000411",
        "servNm": "에너지바우처",
        "jurMnofNm": "산업통상자원부",
        "jurOrgNm": "한국에너지공단",
        "wlfareInfoOutlCn": "냉난방 에너지 구입비를 지원해 취약계층의 계절 부담을 줄이는 사업입니다.",

```

```

108         "sprtTrgtCn": "국민기초생활수급 가구 중 노인·영유아·장애인·임산부 등이 포함된 세대",
109         "slctCritCn": "생계급여 또는 의료급여 수급 가구여야 하며, 세대원 중 더위·추위 민감계층이 있어야 합니다.",
110         "alwServCn": "가구원 수에 따라 연 최대 70만원 상당 바우처 지급",
111         "sprtAmtCn": "연 최대 70만원",
112         "applmetCn": "행정복지센터 방문 신청 또는 복지로 온라인 신청",
113         "applmetUrl": "https://www.energyv.or.kr/",
114         "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000411",
115         "enfcBgngYmd": "20260601",
116         "enfcEndYmd": "20270430",
117         "srvPvsNm": "이용권",
118         "inqNum": "45120"
119     },
120     {
121         "servId": "WII0000633",
122         "servNm": "농어촌 주거환경 개선 융자",
123         "jurMnofNm": "농림축산식품부",
124         "jurOrgNm": "전라남도 순천시",
125         "wlfareInfoOutlCn": "노후 농어촌 주택의 개량과 신축에 필요한 자금을 장기 저리로 융자합니다.",
126         "sprtTrgtCn": "전라남도 순천시에 거주하는 농업인 또는 어업인 세대주",
127         "slctCritCn": "해당 지역에 1년 이상 계속 거주하여야 하며, 주택 노후도 심사를 거쳐 선정합니다.",
128         "alwServCn": "가구당 최대 2억원 융자, 연 2% 고정금리",
129         "sprtAmtCn": "최대 2억원",
130         "applmetCn": "순천시청 농업정책과 방문 신청",
131         "applmetUrl": "https://www.suncheon.go.kr/",
132         "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000633",
133         "enfcBgngYmd": "20260401",
134         "enfcEndYmd": "20261130",
135         "srvPvsNm": "융자",
136         "inqNum": "2210"
137     },
138     {
139         "servId": "WII0000702",
140         "servNm": "서울시 청년월세지원",
141         "jurMnofNm": "서울특별시",
142         "jurOrgNm": "서울특별시",
143         "wlfareInfoOutlCn": "서울에 거주하는 청년 1인 가구의 월세를 지원해 주거비 부담을 낮춥니다.",
144         "sprtTrgtCn": "서울특별시에 주민등록을 둔 만 19세 이상 39세 이하 청년 1인 가구",
145         "slctCritCn": "기준 중위소득 150% 이하이며 임차보증금 8천만원 이하, 월세 60만원 이하 주택에 거주해야 합니다.
국토교통부 청년월세 특별지원과 중복 수혜 불가합니다.",
146         "alwServCn": "월 20만원씩 최대 12개월 지원",
147         "sprtAmtCn": "월 20만원",
148         "applmetCn": "서울주거포털 온라인 신청",
149         "applmetUrl": "https://housing.seoul.go.kr/",
150         "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000702",
151         "enfcBgngYmd": "20260501",
152         "enfcEndYmd": "20260930",
153         "srvPvsNm": "현금",
154         "inqNum": "77410"
155     },
156     {
157         "servId": "WII0000188",
158         "servNm": "아이돌봄서비스",
159         "jurMnofNm": "여성가족부",
160         "jurOrgNm": "여성가족부",
161         "wlfareInfoOutlCn": "가정으로 아이돌보미가 찾아가 아동을 안전하게 돌봐주는 서비스입니다.",
162         "sprtTrgtCn": "만 12세 이하 아동을 양육하는 가정",
163         "slctCritCn": "기준 중위소득 150% 이하 가구는 이용요금의 일부를 정부가 지원합니다. 맞벌이·다자녀 가정을 우선
선정합니다.",
164         "alwServCn": "시간제·영아종일제 돌봄 서비스 및 이용요금 차등 지원",
165         "sprtAmtCn": "시간당 최대 12,180원 지원",
166         "applmetCn": "아이돌봄서비스 누리집 온라인 신청",
167         "applmetUrl": "https://www.idolbom.go.kr/",
168         "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000188",

```

```

169         "enfcBgngYmd": "20260101",
170         "enfcEndYmd": "",
171         "srvPvsNm": "서비스",
172         "inqNum": "412009"
173     },
174     {
175         "servId": "WII0000520",
176         "servNm": "청년 마음건강 지원사업",
177         "jurMnofNm": "보건복지부",
178         "jurOrgNm": "보건복지부",
179         "wlfareInfoOutlCn": "정서적 어려움을 겪는 청년에게 전문 심리상담 서비스를 제공합니다.",
180         "sprtTrgtCn": "만 19세 이상 34세 이하 청년",
181         "slctCritCn": "소득 기준은 적용하지 않으며 자립준비청년과 보호연장아동을 우선 지원합니다.",
182         "alwServCn": "1회 50분 전문 심리상담 10회 제공",
183         "sprtAmtCn": "",
184         "applmetCn": "복지로 온라인 신청",
185         "applmetUrl": "https://www.bokjiro.go.kr/",
186         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000520",
187         "enfcBgngYmd": "20260201",
188         "enfcEndYmd": "20261215",
189         "srvPvsNm": "서비스",
190         "inqNum": "60112"
191     },
192     {
193         "servId": "WII0000861",
194         "servNm": "해운대구 저소득 한부모가족 생활안정 지원",
195         "jurMnofNm": "부산광역시 해운대구",
196         "jurOrgNm": "부산광역시 해운대구",
197         "wlfareInfoOutlCn": "한부모가족의 생활 안정과 자녀 양육을 돕기 위한 구 자체 지원사업입니다.",
198         "sprtTrgtCn": "부산광역시 해운대구에 거주하는 한부모가족 세대주",
199         "slctCritCn": "기준 중위소득 60% 이하 가구를 대상으로 하며, 관내 6개월 이상 거주해야 합니다.",
200         "alwServCn": "가구당 연 60만원 생활안정자금 지급",
201         "sprtAmtCn": "연 60만원",
202         "applmetCn": "해운대구청 가족복지과 방문 신청",
203         "applmetUrl": "https://www.haeundae.go.kr/",
204         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000861",
205         "enfcBgngYmd": "20260301",
206         "enfcEndYmd": "20261031",
207         "srvPvsNm": "현금",
208         "inqNum": "3390"
209     },
210     {
211         "servId": "WII0000970",
212         "servNm": "국민취업지원제도 (I유형)",
213         "jurMnofNm": "고용노동부",
214         "jurOrgNm": "고용노동부",
215         "wlfareInfoOutlCn": "취업을 원하는 구직자에게 취업지원 서비스와 생계 지원을 함께 제공하는 한국형
실업부조입니다.",
216         "sprtTrgtCn": "만 15세 이상 69세 이하 구직자",
217         "slctCritCn": "가구 기준 중위소득 60% 이하이면서 재산 4억원 이하여야 하며, 최근 2년 내 100일 이상 취업 경험이
있어야 합니다.",
218         "alwServCn": "구직촉진수당 월 50만원을 최대 6개월 지급하고 취업지원 서비스를 연계",
219         "sprtAmtCn": "월 50만원",
220         "applmetCn": "국민취업지원제도 누리집 온라인 신청 또는 고용센터 방문",
221         "applmetUrl": "https://www.kua.go.kr/",
222         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000970",
223         "enfcBgngYmd": "20260101",
224         "enfcEndYmd": "",
225         "srvPvsNm": "현금",
226         "inqNum": "540221"
227     }
228 ]
229 }
230 }

```

```
231     }
232 }
```

5.2 지원사업 공고 응답 구조

ingest/fixtures/bizinfo.json

136 lines · JSON

```
1  {
2    "_fixture_note": "기업마당 지원사업 API 응답 봉투 흉내. 필드명·구조는 W1 실물 검증 대상 (SPEC §3.3). inquireCo(조회수)
   는 휘발성 필드로 content_hash 에서 제외된다 (SPEC §3.2).",
3    "resultCode": "0000",
4    "resultMsg": "정상",
5    "totalCount": 8,
6    "jsonArray": [
7      {
8        "pblancId": "PBLN_0000000000098431",
9        "pblancNm": "2026년 소상공인 정책자금 (일반경영안정자금)",
10       "jrdsInstNm": "중소벤처기업부",
11       "excInstNm": "소상공인시장진흥공단",
12       "bsnsSumryCn": "경영 애로를 겪는 소상공인에게 운전자금을 저리로 직접 대출하여 경영 안정을 돕습니다. 대출한도는 업체당 최대
   7천만원이며 금리는 정책자금 기준금리에 연동됩니다.",
13       "trgetNm": "상시근로자 5인 미만 소상공인 또는 자영업자로서 사업자등록을 마친 자. 업력 3년 이상 업체를 우선 지원하며,
   신용등급 하위 구간은 별도 심사합니다.",
14       "pldirSportRealmLclasCodeNm": "금융",
15       "reqstBeginEndDe": "20260105 ~ 예산소진시",
16       "sportScvlNm": "업체당 최대 7,000만원",
17       "reqstMthPapersCn": "소상공인정책자금 누리집 온라인 신청 후 서류 제출",
18       "rceptEnghmpgUrl": "https://ols.semas.or.kr/",
19       "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
   pblancId=PBLN_0000000000098431",
20       "inquireCo": "128441",
21       "creatPnttm": "2026-01-05 09:00:00"
22     },
23     {
24       "pblancId": "PBLN_0000000000099022",
25       "pblancNm": "부산 지역신용보증재단 소상공인 특례보증",
26       "jrdsInstNm": "부산광역시",
27       "excInstNm": "부산신용보증재단",
28       "bsnsSumryCn": "담보력이 부족한 지역 소상공인의 은행 대출에 대해 보증서를 발급하여 자금 조달을 돕는 사업입니다. 보증료
   일부를 부산광역시가 지원합니다.",
29       "trgetNm": "부산광역시 해운대구에 사업장을 둔 소상공인. 사업자등록 후 6개월 이상 경과하고 관내 6개월 이상 계속 거주 또는
   사업장을 유지해야 합니다.",
30       "pldirSportRealmLclasCodeNm": "금융",
31       "reqstBeginEndDe": "20260201 ~ 20261231",
32       "sportScvlNm": "업체당 최대 5,000만원",
33       "reqstMthPapersCn": "부산신용보증재단 영업점 방문 접수",
34       "rceptEnghmpgUrl": "https://www.busanshinbo.or.kr/",
35       "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
   pblancId=PBLN_0000000000099022",
36       "inquireCo": "9120",
37       "creatPnttm": "2026-02-01 09:00:00"
38     },
39     {
40       "pblancId": "PBLN_0000000000097355",
41       "pblancNm": "노란우산공제 신규가입 장려금 지원",
42       "jrdsInstNm": "중소벤처기업부",
43       "excInstNm": "중소기업중앙회",
44       "bsnsSumryCn": "폐업·노령 등에 대비한 소상공인 공제 가입을 촉진하기 위해 신규 가입자에게 장려금을 지급합니다.",
45       "trgetNm": "노란우산공제에 신규 가입한 소상공인 및 개인사업자",
46       "pldirSportRealmLclasCodeNm": "금융",
47       "reqstBeginEndDe": "20260101 ~ 20261231",
48       "sportScvlNm": "월 2만원씩 12개월",
49       "reqstMthPapersCn": "노란우산 누리집 온라인 신청",
50       "rceptEnghmpgUrl": "https://www.8899.or.kr/",
```

```

51      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000097355",
52      "inqireCo": "31204",
53      "creatPnttm": "2026-01-02 09:00:00"
54  },
55  {
56      "pblancId": "PBLN_000000000098770",
57      "pblancNm": "청년전용 창업기업 지원자금",
58      "jrdsInsttNm": "중소벤처기업부",
59      "excInsttNm": "중소벤처기업진흥공단",
60      "bsnsSumryCn": "우수한 아이디어를 가진 청년 창업자에게 창업 초기 운전자금과 시설자금을 융자 지원합니다.",
61      "trgetNm": "만 39세 이하 예비창업자 또는 창업 3년 이내 기업의 대표자. 창업 교육 이수자에 한하여 신청할 수 있습니다.",
62      "pldirSportRealmlclasCodeNm": "창업",
63      "reqstBeginEndDe": "20260310 ~ 20260430",
64      "sportScvlNm": "기업당 최대 1억원",
65      "reqstMthPapersCn": "중소벤처기업진흥공단 온라인 신청",
66      "rceptEnghmpgUrl": "https://www.kosmes.or.kr/",
67      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000098770",
68      "inqireCo": "48800",
69      "creatPnttm": "2026-03-10 09:00:00"
70  },
71  {
72      "pblancId": "PBLN_000000000099410",
73      "pblancNm": "전통시장 상인 경영개선 컨설팅",
74      "jrdsInsttNm": "중소벤처기업부",
75      "excInsttNm": "소상공인시장진흥공단",
76      "bsnsSumryCn": "전통시장 점포의 매출 부진 원인을 진단하고 상품 구성, 진열, 홍보 전략을 컨설팅합니다.",
77      "trgetNm": "전통시장 및 상점이 내에서 영업 중인 소상공인",
78      "pldirSportRealmlclasCodeNm": "경영",
79      "reqstBeginEndDe": "20260401 ~ 20261120",
80      "sportScvlNm": "점포당 최대 300만원 상당",
81      "reqstMthPapersCn": "소상공인시장진흥공단 지역센터 방문 접수",
82      "rceptEnghmpgUrl": "https://www.semas.or.kr/",
83      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000099410",
84      "inqireCo": "7412",
85      "creatPnttm": "2026-04-01 09:00:00"
86  },
87  {
88      "pblancId": "PBLN_000000000096001",
89      "pblancNm": "여성기업 확인 및 판로지원 사업",
90      "jrdsInsttNm": "중소벤처기업부",
91      "excInsttNm": "한국여성경제인협회",
92      "bsnsSumryCn": "여성기업 확인서를 발급하고 공공기관 우선구매 및 전시회 참가를 지원합니다.",
93      "trgetNm": "여성기업 확인서를 보유하고 있거나 발급 신청 중인 중소기업. 대표자 성별과 무관하게 법인 요건을 충족하면 신청할 수
있습니다.",
94      "pldirSportRealmlclasCodeNm": "판로",
95      "reqstBeginEndDe": "20260201 ~ 20261130",
96      "sportScvlNm": "기업당 최대 500만원",
97      "reqstMthPapersCn": "한국여성경제인협회 온라인 신청",
98      "rceptEnghmpgUrl": "https://www.womanbiz.or.kr/",
99      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000096001",
100      "inqireCo": "5321",
101      "creatPnttm": "2026-02-01 09:00:00"
102  },
103  {
104      "pblancId": "PBLN_000000000099880",
105      "pblancNm": "스마트상점 기술보급 사업",
106      "jrdsInsttNm": "중소벤처기업부",
107      "excInsttNm": "소상공인시장진흥공단",
108      "bsnsSumryCn": "키오스크, 스마트 주문 시스템 등 디지털 기술 도입 비용의 일부를 지원해 소상공인의 운영 효율을 높입니다.",
109      "trgetNm": "사업자등록을 마친 소상공인으로 오프라인 점포를 운영 중인 자",
110      "pldirSportRealmlclasCodeNm": "기술",
111      "reqstBeginEndDe": "20260415 ~ 20260630",
112      "sportScvlNm": "점포당 최대 500만원",

```

```

113     "reqstMthPapersCn": "스마트상점 누리집 온라인 신청",
114     "rceptEngnHmpgUrl": "https://www.smartsangjum.kr/",
115     "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000099880",
116     "inqireCo": "22190",
117     "creatPnttm": "2026-04-15 09:00:00"
118 },
119 {
120     "pblancId": "PBLN_000000000095220",
121     "pblancNm": "재도전 성공패키지 (재창업 지원)",
122     "jrdsInsttNm": "중소벤처기업부",
123     "excInsttNm": "창업진흥원",
124     "bsnsSumryCn": "폐업 경험이 있는 재창업자에게 사업화 자금과 교육, 멘토링을 함께 제공합니다.",
125     "trgtetNm": "예비 또는 3년 이내 재창업자. 폐업 이력이 확인되어야 하며 채무불이행 상태인 경우 신청이 제한됩니다.",
126     "pldirSportRealmLclasCodeNm": "창업",
127     "reqstBeginEndDe": "20260220 ~ 20260331",
128     "sportScvlNm": "최대 6,000만원",
129     "reqstMthPapersCn": "K-스타트업 누리집 온라인 신청",
130     "rceptEngnHmpgUrl": "https://www.k-startup.go.kr/",
131     "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000095220",
132     "inqireCo": "16740",
133     "creatPnttm": "2026-02-20 09:00:00"
134 }
135 ]
136 }

```

5.3 금융상품 공시 응답 구조

ingest/fixtures/finlife.json

135 lines · JSON

```

1  {
2    "_fixture_note": "금감원 finlife 예·적금 공시 API 응답 봉투 흉내. baseList + optionList 를 fin_prdt_cd 로 조인하는
구조까지 재현했다. 필드명은 W1 실물 검증 대상 (SPEC S3.3). fin_co_subm_day 는 매 공시 갱신되는 휘발성 필드라 content_hash
에서 제외된다.",
3    "result": {
4      "prdt_div": "S",
5      "total_count": 7,
6      "max_page_no": 1,
7      "now_page_no": 1,
8      "err_cd": "000",
9      "err_msg": "정상처리 되었습니다.",
10     "baseList": [
11       {
12         "dcls_month": "202608",
13         "fin_co_no": "0010001",
14         "fin_prdt_cd": "KB-YOUTH-01",
15         "kor_co_nm": "국민은행",
16         "fin_prdt_nm": "KB 청년도약 우대적금",
17         "join_way": "인터넷,스마트폰,영업점",
18         "mtrt_int": "만기 후 1년 이내 약정이율의 50%",
19         "spcl_cnd": "급여이체 실적 보유 시 연 0.5%p, 자동이체 6회 이상 시 연 0.3%p 우대",
20         "join_deny": "1",
21         "join_member": "만 19세 이상 34세 이하 실명의 개인. 기준 중위소득 180% 이하 가구에 한합니다.",
22         "etc_note": "1인 1계좌, 월 납입한도 70만원",
23         "max_limit": "70000000",
24         "dcls_strt_day": "20260801",
25         "fin_co_subm_day": "202608031030"
26       },
27       {
28         "dcls_month": "202608",
29         "fin_co_no": "0010002",
30         "fin_prdt_cd": "IBK-BOSS-02",
31         "kor_co_nm": "기업은행",

```

```

32     "fin_prdt_nm": "IBK 사장님 우대적금",
33     "join_way": "영업점,스마트폰",
34     "mtrt_int": "만기 후 기본이율 적용",
35     "spcl_cnd": "노란우산공제 가입 고객 연 0.4%p, 카드 가맹점 실적 보유 시 연 0.3%p 우대",
36     "join_deny": "1",
37     "join_member": "사업자등록증을 보유한 개인사업자 또는 소상공인",
38     "etc_note": "월 납입한도 300만원, 계약기간 12~36개월",
39     "max_limit": "100000000",
40     "dcls_strt_day": "20260801",
41     "fin_co_subm_day": "202608031102"
42 },
43 {
44     "dcls_month": "202608",
45     "fin_co_no": "0010003",
46     "fin_prdt_cd": "SHB-EDEP-05",
47     "kor_co_nm": "신한은행",
48     "fin_prdt_nm": "신한 e-정기예금",
49     "join_way": "인터넷,스마트폰",
50     "mtrt_int": "만기 후 1개월 이내 약정이율의 50%",
51     "spcl_cnd": "해당사항 없음",
52     "join_deny": "1",
53     "join_member": "실명의 개인 및 개인사업자",
54     "etc_note": "최소 가입금액 100만원",
55     "max_limit": "500000000",
56     "dcls_strt_day": "20260801",
57     "fin_co_subm_day": "202608030940"
58 },
59 {
60     "dcls_month": "202608",
61     "fin_co_no": "0010004",
62     "fin_prdt_cd": "NH-SENIOR-03",
63     "kor_co_nm": "농협은행",
64     "fin_prdt_nm": "NH 시니어 행복적금",
65     "join_way": "영업점,인터넷",
66     "mtrt_int": "만기 후 기본이율의 50%",
67     "spcl_cnd": "연금 수령 실적 보유 시 연 0.6%p, 농업인 확인서 제출 시 연 0.2%p 우대",
68     "join_deny": "1",
69     "join_member": "만 60세 이상 실명의 개인",
70     "etc_note": "월 납입한도 50만원",
71     "max_limit": "300000000",
72     "dcls_strt_day": "20260801",
73     "fin_co_subm_day": "202608041500"
74 },
75 {
76     "dcls_month": "202608",
77     "fin_co_no": "0010005",
78     "fin_prdt_cd": "HANA-FREE-07",
79     "kor_co_nm": "하나은행",
80     "fin_prdt_nm": "하나 청년 자유적금",
81     "join_way": "스마트폰",
82     "mtrt_int": "만기 후 1년 이내 약정이율의 30%",
83     "spcl_cnd": "첫 거래 고객 연 0.5%p, 마케팅 동의 시 연 0.1%p 우대",
84     "join_deny": "1",
85     "join_member": "만 19세 이상 34세 이하 실명의 개인",
86     "etc_note": "자유적립식, 월 최대 50만원",
87     "max_limit": "18000000",
88     "dcls_strt_day": "20260801",
89     "fin_co_subm_day": "202608031345"
90 },
91 {
92     "dcls_month": "202608",
93     "fin_co_no": "0010006",
94     "fin_prdt_cd": "WOORI-HOUSE-09",
95     "kor_co_nm": "우리은행",
96     "fin_prdt_nm": "우리 주택청약 우대 정기예금",
97     "join_way": "영업점,인터넷,스마트폰",
98     "mtrt_int": "만기 후 기본이율 적용",

```

```

99     "spl_cnd": "무주택 세대주 확인서 제출 시 연 0.4%p, 청약저축 보유 시 연 0.2%p 우대",
100     "join_deny": "1",
101     "join_member": "실명의 개인",
102     "etc_note": "가입금액 300만원 이상",
103     "max_limit": "300000000",
104     "dcls_strt_day": "20260801",
105     "fin_co_subm_day": "202608030855"
106 },
107 {
108     "dcls_month": "202608",
109     "fin_co_no": "0010007",
110     "fin_prdt_cd": "KAKAO-PARK-11",
111     "kor_co_nm": "카카오뱅크",
112     "fin_prdt_nm": "카카오뱅크 세이프박스",
113     "join_way": "스마트폰",
114     "mtrt_int": "해당사항 없음",
115     "spl_cnd": "해당사항 없음",
116     "join_deny": "1",
117     "join_member": "실명의 개인",
118     "etc_note": "입출금 자유, 한도 내 일복리",
119     "max_limit": "100000000",
120     "dcls_strt_day": "20260801",
121     "fin_co_subm_day": "202608031201"
122 }
123 ],
124 "optionList": [
125     { "fin_co_no": "0010001", "fin_prdt_cd": "KB-YOUTH-01", "intr_rate_type_nm": "단리", "save_trm":
126 "12", "intr_rate": 3.5, "intr_rate2": 4.8 },
127     { "fin_co_no": "0010001", "fin_prdt_cd": "KB-YOUTH-01", "intr_rate_type_nm": "단리", "save_trm":
128 "24", "intr_rate": 3.7, "intr_rate2": 5.0 },
129     { "fin_co_no": "0010002", "fin_prdt_cd": "IBK-BOSS-02", "intr_rate_type_nm": "단리", "save_trm":
130 "12", "intr_rate": 3.2, "intr_rate2": 3.9 },
131     { "fin_co_no": "0010003", "fin_prdt_cd": "SHB-EDEP-05", "intr_rate_type_nm": "단리", "save_trm":
132 "12", "intr_rate": 3.0, "intr_rate2": 3.0 },
133     { "fin_co_no": "0010004", "fin_prdt_cd": "NH-SENIOR-03", "intr_rate_type_nm": "단리", "save_trm":
134 "12", "intr_rate": 3.3, "intr_rate2": 4.1 },
135     { "fin_co_no": "0010005", "fin_prdt_cd": "HANA-FREE-07", "intr_rate_type_nm": "단리", "save_trm":
136 "12", "intr_rate": 3.1, "intr_rate2": 3.7 },
137     { "fin_co_no": "0010006", "fin_prdt_cd": "WOORI-HOUSE-09", "intr_rate_type_nm": "단리", "save_trm":
138 "12", "intr_rate": 2.9, "intr_rate2": 3.5 },
139     { "fin_co_no": "0010007", "fin_prdt_cd": "KAKAO-PARK-11", "intr_rate_type_nm": "단리", "save_trm":
140 "0", "intr_rate": 2.0, "intr_rate2": 2.0 }
141 ]
142 }
143 }

```

6. 자격 판정 기준 데이터 (JSON)

API 응답의 서술형 자격요건을 정형 조건으로 변환할 때 사용하는 기준 데이터입니다. 행정표준코드(법정동코드)와 통계청 자료를 근거로 구성했습니다.

6.1 지역 코드

shared/regions.json61 lines · JSON

```
1 {
2   "_note": "행정표준코드 기반. 시도 2자리 / 시군구 5자리 (SPEC §5 regions 규약). 시군구는 MVP 데모용 부분집합 - W1에 전량 (~250건) 반영 예정.",
3   "sido": [
4     { "code": "11", "name": "서울특별시" },
5     { "code": "26", "name": "부산광역시" },
6     { "code": "27", "name": "대구광역시" },
7     { "code": "28", "name": "인천광역시" },
8     { "code": "29", "name": "광주광역시" },
9     { "code": "30", "name": "대전광역시" },
10    { "code": "31", "name": "울산광역시" },
11    { "code": "36", "name": "세종특별자치시" },
12    { "code": "41", "name": "경기도" },
13    { "code": "43", "name": "충청북도" },
14    { "code": "44", "name": "충청남도" },
15    { "code": "46", "name": "전라남도" },
16    { "code": "47", "name": "경상북도" },
17    { "code": "48", "name": "경상남도" },
18    { "code": "50", "name": "제주특별자치도" },
19    { "code": "51", "name": "강원특별자치도" },
20    { "code": "52", "name": "전북특별자치도" }
21  ],
22  "sigungu": [
23    { "code": "11110", "name": "종로구", "sido": "11" },
24    { "code": "11140", "name": "중구", "sido": "11" },
25    { "code": "11170", "name": "용산구", "sido": "11" },
26    { "code": "11215", "name": "광진구", "sido": "11" },
27    { "code": "11230", "name": "동대문구", "sido": "11" },
28    { "code": "11260", "name": "종량구", "sido": "11" },
29    { "code": "11305", "name": "강북구", "sido": "11" },
30    { "code": "11350", "name": "노원구", "sido": "11" },
31    { "code": "11440", "name": "마포구", "sido": "11" },
32    { "code": "11530", "name": "구로구", "sido": "11" },
33    { "code": "11620", "name": "관악구", "sido": "11" },
34    { "code": "11650", "name": "서초구", "sido": "11" },
35    { "code": "11680", "name": "강남구", "sido": "11" },
36    { "code": "11740", "name": "강동구", "sido": "11" },
37    { "code": "26260", "name": "동래구", "sido": "26" },
38    { "code": "26350", "name": "해운대구", "sido": "26" },
39    { "code": "26440", "name": "수영구", "sido": "26" },
40    { "code": "27260", "name": "수성구", "sido": "27" },
41    { "code": "28185", "name": "연수구", "sido": "28" },
42    { "code": "29155", "name": "서구", "sido": "29" },
43    { "code": "30230", "name": "대덕구", "sido": "30" },
44    { "code": "31110", "name": "중구", "sido": "31" },
45    { "code": "36110", "name": "세종시", "sido": "36" },
46    { "code": "41111", "name": "수원시 장안구", "sido": "41" },
47    { "code": "41135", "name": "성남시 분당구", "sido": "41" },
48    { "code": "41285", "name": "고양시 덕양구", "sido": "41" },
49    { "code": "41590", "name": "화성시", "sido": "41" },
50    { "code": "43111", "name": "청주시 상당구", "sido": "43" },
51    { "code": "44131", "name": "천안시 동남구", "sido": "44" },
52    { "code": "46110", "name": "목포시", "sido": "46" },
```

```

53 { "code": "46130", "name": "여주시", "sido": "46" },
54 { "code": "46150", "name": "순천시", "sido": "46" },
55 { "code": "47111", "name": "포항시 남구", "sido": "47" },
56 { "code": "48121", "name": "창원시 의창구", "sido": "48" },
57 { "code": "50110", "name": "제주시", "sido": "50" },
58 { "code": "51110", "name": "춘천시", "sido": "51" },
59 { "code": "52111", "name": "전주시 완산구", "sido": "52" }
60 ]
61 }

```

6.2 직업 분류

shared/occupations.json

17 lines · JSON

```

1 {
2   "_note": "직업 대분류 12종 (SPEC §5 입력 필드 3). code는 DB·파서·화면 공통. synonyms는 파서 사전 록업용 (§6.1).",
3   "categories": [
4     { "code": "employee_office", "name": "사무직·전문직", "synonyms": ["사무직", "회사원", "직장인", "근로자",
5       "전문직"] },
6     { "code": "employee_field", "name": "현장·생산·서비스직", "synonyms": ["생산직", "현장직", "서비스직", "기능공"] },
7     { "code": "self_employed", "name": "자영업·소상공인", "synonyms": ["자영업자", "소상공인", "개인사업자",
8       "사업자", "소기업"] },
9     { "code": "farmer_fisher", "name": "농림어업인", "synonyms": ["농업인", "어업인", "임업인", "농어민", "농가"] },
10    { "code": "freelancer", "name": "프리랜서·특수고용", "synonyms": ["프리랜서", "특수형태근로종사자", "특고",
11      "플랫폼노동자"] },
12    { "code": "student", "name": "학생", "synonyms": ["대학생", "대학원생", "재학생"] },
13    { "code": "jobseeker", "name": "구직자·무직", "synonyms": ["구직자", "무직", "실업자", "미취업", "취업준비생"] },
14    { "code": "homemaker", "name": "주부", "synonyms": ["전업주부", "가사"] },
15    { "code": "public_servant", "name": "공무원·군인", "synonyms": ["공무원", "군인", "직업군인"] },
16    { "code": "medical", "name": "보건·의료", "synonyms": ["의료인", "간호사", "의사", "보건"] },
17    { "code": "education", "name": "교육", "synonyms": ["교사", "교원", "강사", "교직원"] },
18    { "code": "retired", "name": "은퇴·연금생활", "synonyms": ["은퇴자", "퇴직자", "연금수급자", "어르신", "노인"] }
19  ]
20 }

```

6.3 소득분위 기준

shared/income_deciles.json

15 lines · JSON

```

1 {
2   "_note": "소득분위 자가 입력용 라벨 (SPEC §5). 월 소득 구간은 데모용 추정치 - W1에 최신 고시 기준으로 갱신 (연 1회 갱신
3     정책).",
4   "deciles": [
5     { "decile": 1, "label": "1분위", "monthly_income_hint": "월 소득 약 ~95만원" },
6     { "decile": 2, "label": "2분위", "monthly_income_hint": "월 소득 약 95~160만원" },
7     { "decile": 3, "label": "3분위", "monthly_income_hint": "월 소득 약 160~230만원" },
8     { "decile": 4, "label": "4분위", "monthly_income_hint": "월 소득 약 230~300만원" },
9     { "decile": 5, "label": "5분위", "monthly_income_hint": "월 소득 약 300~370만원" },
10    { "decile": 6, "label": "6분위", "monthly_income_hint": "월 소득 약 370~450만원" },
11    { "decile": 7, "label": "7분위", "monthly_income_hint": "월 소득 약 450~540만원" },
12    { "decile": 8, "label": "8분위", "monthly_income_hint": "월 소득 약 540~660만원" },
13    { "decile": 9, "label": "9분위", "monthly_income_hint": "월 소득 약 660~850만원" },
14    { "decile": 10, "label": "10분위", "monthly_income_hint": "월 소득 약 850만원~" }
15  ]
16 }

```

6.4 중위소득 비율 환산표

```
1  {
2    "_note": "기준 중위소득 % → 소득분위 상한 환산표 (SPEC §6.1 midrate_to_decile). '중위소득 X% 이하' 문구에서 X 이하인
첫 행의 decile을 쓴다. 데모용 근사치 - W1에 고시 기준 검증 (연 1회 갱신).",
3    "mapping": [
4      { "max_percent": 50, "decile": 2 },
5      { "max_percent": 75, "decile": 3 },
6      { "max_percent": 100, "decile": 5 },
7      { "max_percent": 120, "decile": 6 },
8      { "max_percent": 150, "decile": 7 },
9      { "max_percent": 180, "decile": 8 },
10     { "max_percent": 200, "decile": 9 },
11     { "max_percent": null, "decile": 10 }
12   ]
13 }
```

7. 이용 준수사항

항목	준수 내용
출처 표시	모든 결과 화면에 데이터 출처 기관명, 원문 URL, 수집 시각을 표시합니다. 별도의 출처 안내 페이지를 운영합니다.
호출 부하	1일 1회 심야 배치. 이용자 요청 시점에는 API를 호출하지 않습니다. 도메인당 1초 1요청, 동시 연결 1개.
재시도	408/429/5xx에 한해 최대 2회, 지수 백오프. 4xx는 즉시 중단하여 무효 요청을 반복하지 않습니다.
인증키 관리	소스코드·저장소에 키를 포함하지 않습니다. 환경변수로만 주입하며 공개 저장소에 커밋되지 않습니다.
데이터 가공	원문을 임의로 수정하지 않습니다. 요약을 생성하는 경우에도 원문 링크를 항상 병기하여 이용자가 원본을 확인할 수 있게 합니다.
개인정보	회원가입·로그인이 없습니다. 입력받은 인적사항은 매칭 연산에만 사용하며 90일 후 자동 삭제합니다. 공공API로 개인정보를 전송하지 않습니다.
영리성	비영리. 공모전 출품 및 공익 목적이며 광고·유료화 계획이 없습니다.
장애 대응	수집 실패 시 직전 성공 데이터를 유지합니다. 빈 결과를 노출하지 않습니다.

본 문서에 포함된 소스코드와 데이터는 github.com/ofseo03/amuguna 에서 확인하실 수 있습니다.
문의: ofseo03@gmail.com